



UPPSALA
UNIVERSITET

IT mDV 26 014

Degree project 30 credits

June, 2026

Comparative Analysis of Methods for Transforming EHR Data to Canonical Graph Elements

Abhinav Ramalingam



UPPSALA
UNIVERSITET

Comparative Analysis of Methods for Transforming EHR Data to Canonical Graph Elements

Abhinav Ramalingam

Abstract

Electronic Health Record (EHR) data is frequently fragmented across siloed systems with varying structures and semantics, which introduces significant challenges for data integration and secondary use. This thesis develops a proof-of-concept pipeline that transforms EHR tables into canonical graph elements, enabling the linking of heterogeneous clinical sources. The research focuses on three primary stages: structural profiling, classifying tables into functional categories such as entities and events, and decomposing complex mixed tables into meaningful substructures. A systematic comparison was conducted between traditional automated methods, including rule-based methods, supervised ML classifiers, and dependency analysis, and Large Language Model (LLM) approaches. Results indicate that while traditional automated approaches achieve strong performance and separability in entity-event classification, LLMs are beneficial for identifying ambiguous mixed tables. Furthermore, for the more complex task of table decomposition, LLM-based approaches significantly outperform purely structural methods by utilising semantic understanding to identify entity boundaries and identifiers that traditional methods often miss. The findings show that while structural signals are largely sufficient for profiling and classification, incorporating semantic reasoning via LLMs is essential for effective identification and decomposition of complex clinical tables.

Faculty of Science and Technology

Uppsala University, Uppsala

Supervisor: Andra-Denis Ionescu Subject reader: Konstantinos Sagonas

Examiner: Lars-Henrik Eriksson

Contents

1	Introduction	1
2	Background	3
2.1	Literature Review	3
2.2	Key Concepts	4
2.2.1	EHR Data	4
2.2.2	Knowledge Graphs	5
2.2.3	Large Language Models	5
3	Methodology	7
3.1	Structural Profiling	7
3.2	Entity-Event Classification	8
3.2.1	Rule-Based Method	8
3.2.2	ML-Based Methods	9
3.2.3	LLM-Based Method	10
3.3	Multi-Entity Table Decomposition	10
3.3.1	Functional Dependency-based column clustering	11
3.3.2	Co-occurrence analysis using inclusion dependencies	12
3.3.3	LLM-guided structural proposals	12
4	Experiments	13
4.1	Data	13
4.1.1	MIMIC-IV	13
4.1.2	e-ICU	14
4.2	Evaluation	14
4.2.1	Classification	14
4.2.2	Decomposition	15
4.3	Experiment methods	15
4.3.1	Profiling	15
4.3.2	Classification	16
4.3.3	Decomposition	18
4.4	Results	18
4.4.1	Classification	18
4.4.2	Decomposition	22
5	Discussion	26
5.1	RQ1: Entity-Event Classification	26
5.2	RQ2: Multi-Entity Table Decomposition	27
5.3	RQ3: Comparison of Traditional and LLM-based Approaches	27
6	Conclusion	29

A	Full Decomposition Evaluations	33
B	LLM Prompts	36
B.1	LLM-Based classification	36
B.1.1	Zero-shot	36
B.1.2	Few-shot with restriction	37
B.2	LLM-Based decomposition	37
B.2.1	Initial prompt	37
B.2.2	Validation prompt	38

List of Figures

4.1	MIMIC Dataset Admissions Table: Comparison plot across decomposition methods . . .	24
4.2	MIMIC Dataset ICUStays Table: Comparison plot across decomposition methods	24
4.3	e-ICU Dataset ApachePredVar Table: Comparison plot across decomposition methods .	25
4.4	e-ICU Dataset Patient Table: Comparison plot across decomposition methods	25

List of Tables

4.1	Comparison of Manual vs SQL-based approaches for every table statistic	16
4.2	Rule-based classification: old rule set	17
4.3	Rule-based classification: refined rule set	17
4.4	Rule-based method: Per-class and aggregated F1-score comparison	18
4.5	Decision tree method: Per-class and aggregated F1-score comparison	19
4.6	Random Forest method: Per-class and aggregated F1-score comparison	20
4.7	Embedding-based method: Per-class and aggregated F1-score comparison	20
4.8	LLM-based method: Per-class and aggregated F1-score comparison - Zero-Shot System Prompt with definitions	21
4.9	LLM-based method: Per-class and aggregated F1-score comparison - Few-Shot System Prompt with definitions and restriction	21
4.10	Random Forest method on e-ICU dataset: Per-class and aggregated F1-score comparison	22
A.1	FD Method on MIMIC: Full Decomposition Evaluation	34
A.2	IND Method on MIMIC: Full Decomposition Evaluation	34
A.3	LLM Method on MIMIC: Full Decomposition Evaluation	35
A.4	LLM Method on e-ICU: Full Decomposition Evaluation	35

Chapter 1

Introduction

Healthcare data is often *fragmented*, meaning it originates from a multitude of heterogeneous sources such as clinical registries, insurance databases, and electronic health records. Data is often *siloed*, meaning it resides in isolated systems that frequently differ in their syntax, structure, and semantics, creating ambiguity between the information model and the underlying domain terminology. These properties introduce challenges in data integration and analysis [Abad-Navarro and Martinez-Costa, 2024].

Health data integration involves fitting into rigid, predefined models such as OMOP (the Observational Medical Outcomes Partnership common data model), which are usually focused on the syntactic structure of data rather than its underlying semantics. Moreover, integration remains a labour-intensive, manual process that calls for defining mappings between source data and target formats. This requires a domain expert with knowledge about medical terminologies and ontologies. Due to the diversity in data formats and standards, the process often results in significant information loss or a lack of guaranteed semantic interoperability between systems. These factors limit the effective reuse of healthcare data in downstream analytical and clinical applications.

As retrospectively collected data is increasingly used in patient care through knowledge discovery [Johnson et al., 2024], a system that understands the semantic layer of the data is highly applicable in both research tasks, such as predictive modelling and enhanced clinical decision support [Manda, 2025]. One of the most common data structures chosen to model real-world concepts is a Knowledge Graph (KG). KGs use a graph representation in which the nodes and edges represent entities or events, and relationships between them, respectively [Dong, 2023][Ehrlinger and Wöß, 2016]. They allow multi-hop traversal, enabling the execution of complex queries on the KG. While preserving the structured nature of data, they also exhibit flexibility by permitting continuous extension of the data structure as real-world knowledge evolves.

Constructing knowledge graphs often remains a manual and error-prone task [Li et al., 2025]. In particular, transforming relational data into graph representations requires identifying entities, relationships, and structure, something that is often done manually. Conventional methods often rely on direct mappings of primary and foreign keys, while more flexible manual designs are costly and require deep expertise in graph modelling Tong et al. [2025]. Recent advances in large language models (LLMs) have enabled their adoption in newer strategies for automating KG creation. However, LLMs are not immune to drawbacks. Firstly, LLMs are prone to hallucinations, which can surface in terms of structural violations, fabricated identifiers, or incorrect data, creating significant integration issues downstream. The non-deterministic nature of LLMs means that identical prompts can produce different outputs across different runs, making the construction process difficult to reproduce. Crucially, purely LLM-based approaches are computationally expensive and can incur high financial costs due to token consumption, especially in large-scale enterprise settings where thousands of entities must be processed.

The challenges associated with data harmonization and LLM usage motivate us to integrate the diverse health data sources into a unified graph representation. Existing medical terminology can be used to recognise and link equivalent medical terms. LLMs can be used to access external medical context, discovering potential links between graph elements, and new connections may be identified using

methods such as graph neural networks. Our thesis focuses on the design of a data transformation pipeline, specifically comparing traditional automated methods and LLM-based approaches for transforming EHR data into canonical graph elements. The overall questions addressed in the research are as follows:

1. How can EHR tables be classified into canonical types - entities and events?
2. How can mixed tables containing both entity and event-level information be identified and decomposed into meaningful substructures?
3. How do traditional automated methods and LLM-based approaches compare across different stages of the pipeline for profiling, classification, and decomposition?

Thesis Outline

The report is organized into 5 main sections, excluding the Introduction, each covering a key aspect of the thesis. Chapter 2, Background, reviews relevant literature pertaining to the identified problem, and provides background information essential to the comprehensive understanding of the thesis. Chapter 3, Methodology, describes the methods employed in the thesis, with subsections corresponding to the specific stages such as profiling, classification, and decomposition. Chapter 4, Experiments, provides descriptions of the datasets used, outlines the evaluation metrics, presents the experiments conducted, and offers a comprehensive analysis of the performance results obtained. Chapter 5, Discussion, ties the observed results back to the research questions. Finally, Chapter 6, Conclusion, synthesizes the findings of the thesis, outlines assumptions and limitations of the thesis, and suggests avenues for future development in the topic.

Chapter 2

Background

This chapter provides the background context for the thesis. The first section reviews existing literature on knowledge graphs, semantic interoperability in healthcare, and the use of large language models and agent-based approaches for graph-oriented data transformation. The second section presents the key concepts required to understand the thesis, including electronic health records, knowledge graphs, and large language models.

2.1 Literature Review

KGs have been created and utilised for a wide variety of applications, such as web search, voice assistants, and recommendation systems, due to their unique advantage to offer both human readability and machine readability. The scale and structure of data have increased in complexity over the decades, or ‘generations’, requiring shifts in the way we think about KGs. Dong [2023] highlights the characteristics of three evolutionary generations of KG development and the techniques developed over time to handle the data complexity. They are, namely, **Entity-based KGs**, defined by rigorous ontologies and minimal ambiguity, **Text-rich KGs** with sparse structured data and abundant ambiguities, and the more recent **Dual-Neural KGs**, which store the explicit knowledge in terms of triples for explainability and implicit knowledge in terms of embeddings for machine understanding. Each generation offers improved scalability, flexibility, and extraction capability, while suffering in terms of factuality and computational cost.

Dong [2023] shows that KGs have achieved significant success in the life sciences, particularly in drug discovery, protein-disease relationships, disease-symptom relationships, and related areas. In fact, healthcare data faces numerous challenges in terms of fragmentation and siloing, difficulty in integrating data at varying levels of detail, and lack of flexibility and interoperability in existing standards, which makes integration difficult. Abad-Navarro and Martinez-Costa [2024] designed and implemented a semantic-driven data harmonization framework that outputs a Semantic Common Data Model (SCDM) built using BioTopLite2, a top-level ontology, which reduces the reliance on any specific interoperability standard, for e.g., FHIR (Fast Healthcare Interoperability Resources). It was achieved by defining mappings based on the meaning of medical concepts using SNOMED CT rather than syntactic field names. SNOMED CT is utilized as a primary reference terminology similar to other standards like ICD-10 or LOINC to provide domain-specific knowledge.

It is increasingly evident that Large Language Models (LLMs) have altered the methodological boundaries of KGs. Tong et al. [2025] designed an LLM-powered framework to automate the conversion of relational schemas to query-optimized graph schemas. Specifically, the paper uses the concept known as LLM agents, which are complex architectures that combine LLMs with key modules such as planning and memory, to better align the design with the user’s intended analytical queries and iteratively refine graph schemas. It can also be said that extending the LLM with planning, memory, and specialized tools overcomes the inherent limitations of pure LLMs, such as hallucinations, token limits, and a lack of late-breaking information. Building on the idea of semantic interoperability, Qiang et al. [2024] designed

an agent-powered framework to align entities between different ontologies. Pure knowledge-based or ML-based systems suffered from being highly labour-intensive or requiring huge amounts of high-quality training data, and pure prompting with LLMs led to unstable results and high token cost, but with limited capabilities. By using Chain-of-Thought (CoT) planning to decompose complex matching tasks into manageable subtasks and hybrid memory with relational database and Retrieval-Augmented Generation (RAG), a simple predictive model turns into an autonomous controller that enables cost-effective search-based matching.

As the landscape of KG creation becomes increasingly dominated by agentic LLM solutions, it creates the need for establishing evaluation metrics to measure specific capabilities of KG creation. Mihindukulasooriya et al. [2023] designed an evaluation framework specifically to extract facts from text data while strictly adhering to a provided ontology. The framework uses seven distinct metrics to evaluate model performance across three categories, namely, Fact Extraction Accuracy with Standard Precision (P), Recall (R), F1 score, to compare generated triples against the ground truth, Ontology Conformance (OC) to measure the percentage of output triples where the relation belongs to the canonical relations listed in the ontology, and Subject (SH), Relation (RH), and Object (OH) hallucinations by checking if generated terms exist in the input sentence or ontology.

The theory and methods of KG creation have undergone significant advancements with the advent of newer approaches involving LLMs and Autonomous Agents. However, the field and domain, when studied together, reveal different challenges around reliability, scalability, and structural consistency in graph creation. Firstly, developing ontologies is an expert-driven process that is labour-intensive and time-consuming. Manually defining precise relationships and maintaining logical consistency takes months or years for major clinical resources such as the Gene Ontology [Qiang et al., 2024]. Even in modern frameworks such as the SCDM [Abad-Navarro and Martinez-Costa, 2024], defining initial mappings between source data and the ontology remains the most time-consuming manual task.

Further, LLMs frequently generate syntactically sound but factually incorrect responses [Qiang et al., 2024], which in the medical domain manifests as the fabrication of non-existent clinical identifiers, e.g., fake SNOMED or DOID codes, or the assertion of spurious relationships between genes and diseases that lack literature support. LLMs exhibit "shallow" reasoning because they lack the rigorous internal consistency checks found in formal logic systems, and because foundation models are "frozen" at their training cutoff, they cannot absorb new medical discoveries or evolving clinical guidelines without expensive retraining or complex Retrieval-Augmented Generation (RAG) pipelines. Fine-tuning LLMs on sensitive healthcare data risks leaking private clinical information, and LLMs often tend to function as black boxes, meaning there is a critical need for explainable approaches that provide an auditable path for every clinical insight or decision-support suggestion [Dong, 2023].

Another fundamental aspect is the lack of domain-specific benchmarks to measure model performance on large medical ontologies. Most existing benchmarks, such as *Text2KGBench* [Mihindukulasooriya et al., 2023], intentionally use smaller-sized ontologies to accommodate the token limits of LLMs, which does not reflect the massive scale of clinical ontologies. There is a research gap in establishing "High-Utility" metrics that measure the actual burden on human experts, such as the "Identifier Fidelity Rate", correct grounding to canonical IDs, or "Curator Minutes per Accepted Edit," which quantify how much an LLM agent actually accelerates the clinical workflow versus how much time is wasted correcting its hallucinations.

2.2 Key Concepts

2.2.1 EHR Data

Electronic Health Records (EHRs) are clinical information systems that collect, store, and present longitudinal electronic data during the delivery of healthcare [Kim et al., 2019]. Since the use of hieroglyphics and papyrus in the early 1,600-3,000 BC, followed by a steady use of paper medical records since the 1900s, the age of computer technology paved the way for EHRs [Evans, 2016]. The most common data

formats used in EHRs are either structured and encoded, structured with no encoding, or unstructured data with machine-readable text or scanned text, such as faxes and PDFs [Kim et al., 2019].

The United States has almost universally adopted EHRs, with more than 95% of hospitals that have incorporated hospital-wide and intensive care unit (ICU) modules [Henry et al., 2016]. EHRs are utilized by clinicians and admin systems to document medical histories and prescribe medications [Evans, 2016], billing and scheduling [Kim et al., 2019], and also by researchers who use EHR data for purposes such as epidemiology, which is a study of health and disease conditions across a defined population.

An advantageous aspect of EHRs is that they enable accessibility to patient records, helping patients optimize their personal health goals or lifestyle as given in the documentation. Further, large-scale analysis of EHR data enables applications to identify disease patterns, temporal trends, and correlations [Evans, 2016]. In recent times, EHRs have evolved from a pure patient data representation system to a complex information system that can be used to extract various insights [Peters and Buntrock, 2014]. This includes clinical decision support, predictive modelling and forecasting [Evans, 2016], regulatory decisions [Kim et al., 2019], and knowledge discovery and semantic exploration [Abad-Navarro and Martinez-Costa, 2024].

2.2.2 Knowledge Graphs

Knowledge graphs have been a prevalent technology for representing and using knowledge across various domains and applications [Li et al., 2025]. They exhibit a significant role in industry by powering search engines, voice assistants, and recommendation systems for major technology companies, including Google, Microsoft, Amazon, and Meta [Dong, 2023]. One of the most commonly referred definitions of a knowledge graph is authored by Ehrlinger and Wöß [Ehrlinger and Wöß, 2016], which states: *A knowledge graph acquires and integrates information into an ontology and applies a reasoner to derive new knowledge.* As a component of the definition, an ontology can be defined as a formal, explicit specification of a shared conceptualization that provides a standardized, computable representation of domain knowledge [Qiang et al., 2024]. It contains the schema for a knowledge graph, including entity classes often organized in a hierarchy or taxonomy, relationships between those classes, and logical rules or constraints that govern them [Qiang et al., 2024][Dong, 2023]. An ontology serves as a blueprint for transforming fragmented relational data into a unified, hierarchical structure that can incorporate new knowledge without disrupting the core system [Abad-Navarro and Martinez-Costa, 2024].

KGs serve as centralized repositories that provide formal, computable representations of domain knowledge, enabling structured integration and reasoning [Li et al., 2025]. In structural terms, a KG is a knowledge base that utilizes a graph representation, modelling real-world entities as vertices and relationships as edges, often enriched with specific properties. The methodology for constructing KGs has evolved through several generations, transitioning from traditional, formulaic models to advanced semantic approaches [Dong, 2023]. Constructing an effective KG requires balancing the syntactic regularity of the input data with its contextual and semantic complexity, which ultimately determines whether structural extraction or deep inferential reasoning is the most useful path.

2.2.3 Large Language Models

Large Language Models (LLMs) are transformer-based neural language models containing billions of parameters trained on massive, diverse text corpora [Yang et al., 2023]. It is often argued that LLMs, similar to neural networks, encode knowledge by relying on statistical patterns of language rather than on explicit understanding and reasoning [Bender and Koller, 2020]. LLMs have been shown to be useful in a wide variety of downstream NLP tasks due to the vast number of parameters and huge datasets used to pre-train them. Recent breakthroughs in large language models are primarily attributed to the transformer architecture. LLMs generally utilise a pre-training objective centred on 'next-token modelling,' which is an autoregressive process where the system is trained to predict future or missing tokens given the previous context. It has also been shown in a number of cases that prompting, or

providing a specific instruction to the LLM to guide it towards a particular output, enhances the accuracy of responses [Naveed et al., 2025].

Furthermore, LLMs can generalize across domains without explicit feature engineering [Brown et al., 2020] and can use context to infer structure from implicitly embedded information. In the healthcare domain, LLMs are reshaping healthcare delivery and research across diverse areas. The use of LLMs is being explored in multiple healthcare applications, notably pre-consultation and self-assessment tools, EHR summarization and assistance, text-based assistance in cognitive behavioural therapy and emotion regulation, and education and research tasks [Yang et al., 2023].

In spite of the utility and versatility offered by LLMs, several critical factors must be kept in mind, especially while using them for high-stakes applications such as healthcare. These include risk of leaking confidential patient information, managing hallucinations, which are fabricated or incorrect information, and understanding biases in training data. The 'black-box' nature of LLMs means their decision-making processes are often opaque, which may also pose challenges worth addressing [Naveed et al., 2025][Yang et al., 2023].

Chapter 3

Methodology

A knowledge graph may have one or many elements, namely:

- **Entities**, which represent things that exist in space, as opposed to things that happen. *Patient*, *provider*, *location*, and *device* are examples of entity tables.
- **Events**, which represent things that happen at a specific time and usually have a start time and/or an end time. They usually have one or many references to an entity, such as *patient* or *provider*. Examples of event tables include *admission*, *diagnosis*, *lab events*, and *procedure* tables.
- **Relationships**, which are connections that link entities or events in the domain.
- **Properties**, which are characteristics that are used to describe an entity or event in the domain.

Constructing a knowledge graph from EHR data requires extracting these elements and mapping them into a structured representation. Our pipeline, designed to transform input EHR data into graph representations, aims to classify the tables into canonical types: entities and events. We apply profiling techniques to the tables to extract features such as column data types, uniqueness ratios, null patterns, and candidate primary and foreign keys, which serve as input for classification.

An observation from the data reveals certain tables that contain both entity-level attributes and event-level information. They usually contain an entity with timestamps and other attributes from different entities. For example, in the MIMIC-IV dataset [Johnson et al., 2024], the *admissions* table contains information about admission events, which has starttime and endtime columns, along with attributes from the patient entity. We call them "mixed" tables. As part of our pipeline, we decompose mixed tables into meaningful substructures that can be further mapped to entities and events

3.1 Structural Profiling

We designed the profiler to extract various structural signals selected from the tables and output structured metadata, or table 'profiles'. The profile includes column data types, uniqueness ratios, null patterns, candidate primary keys, inclusion dependencies, which are candidate foreign keys, and timestamp signals. Using the inferred data types from any method, we defined timestamp signals as those columns whose data types were either DATE or TIMESTAMP. The highest uniqueness column with zero null ratio and which is not a timestamp, is selected as the candidate primary key.

An inclusion dependency (IND) is a structural relationship in relational databases where all values in one column, or combination of columns, are a subset of the values in another column, or combination of columns. For extracting INDs, we considered a manual brute-force approach. However, the approach involves checking set containment over every column pair across all the tables in the dataset, along with checking the existence of every element in the first column within the second column for each column pair. This makes the computation infeasible for large multi-table clinical datasets. So we considered two optimized methods:

- **Single Pass Inclusion DEpendency Recognition (SPIDER)**: This algorithm performs a parallel, single-pass check across all candidate pairs by using a min-heap to keep multiple pointers aligned across sorted attribute files. [Bauckmann et al., 2007]
- **Fast Approximate IND Discovery Algorithm, (FAIDA)**: This is a hybrid approximate method that uses HyperLogLog for high-cardinality columns and sampled inverted indices for low-cardinality columns [Kruse et al., 2017]. HyperLogLog is a probabilistic algorithm used to estimate the number of distinct values in a column, enabling efficient processing.

We observe that both FAIDA and SPIDER generate large sets of candidate INDs, including many that are syntactically valid but not meaningful. To address this, we apply a filtering step that removes noisy or non-informative INDs based on filters such as type compatibility, primary key checks, and unary IND checks. We allow the filters themselves to be modifiable because of possible differences between dataset characteristics and profiling goals.

The profiling stage operates under the assumption that the selected structural features sufficiently capture the relevant information contained in a table. This may limit the expressiveness of the profiling representation, and richer or more diverse profiling information could potentially improve downstream performance by providing additional structural context to the models.

3.2 Entity-Event Classification

Once we generate the profiles, table classification determines how a table is positioned in the knowledge graph. We designed different methods to classify tables into entity, event, and mixed types. Each type is characterised by the structure of the table and the data records it contains. The different methods designed to classify the tables are:

- Rule-based classification using rules that are defined manually.
- Supervised ML classification using profiling features
- LLM-assisted classification with different inputs such as columns, basic schema input, full profile, and data samples, and prompting strategies such as zero-shot and few-shot.

3.2.1 Rule-Based Method

We designed the rule-based method to input a table profile and a rule set and output one of the three types: entity, event, mixed. The rule set contains a list of rules where each rule is a list of conditions for a table to match, with corresponding confidence scores for entity, event and mixed. Here, the confidence scores are manually set values. The conditions used in our rule-based method include:

- number of columns
- number of timestamp columns
- number of foreign-key candidates
- presence of primary-key candidates

Each rule in the rule set are a subset of the conditions checked. Multiple rules can match for a single input table. From the rule set, the matched rules are extracted and their corresponding confidence scores are averaged across each type, entity, event and mixed, to output the final confidence scores. If no rules were matched, the method outputs default confidence scores which is 0.34 for entity, 0.33 for event, and 0.33 for mixed. We then classify the table with type having the highest confidence score.

3.2.2 ML-Based Methods

We designed different traditional ML approaches using either statistical or semantic input representations from the input data. Statistical representations encode numerical characteristics of tables through features such as row and column counts, timestamp frequencies, foreign-key candidates, and primary-key presence. Semantic representations capture the meaning of the tables from table names and column names. For the statistical approaches, we applied data augmentation to address class imbalance. We did this by creating new mixed table profiles from existing entity and event tables, and some entity and event profiles from existing mixed tables. We then approximated the statistics of the new synthetic tables.

For the statistical feature-based approaches, we selected decision tree and random forest classifiers due to their suitability for structured tabular data. The tables were manually annotated with entity, event, and mixed labels. We built a feature extraction module that loads the table profiles and annotation labels to construct a feature dataset for ML classification. The extracted features, which are the inputs for our statistical approaches, are as follows:

- `log_rows`: Computed logarithm of row count, which reduces scale differences between very small and very large tables.
- `ts_ratio`: Ratio of timestamp columns to total columns. Indicates the dominance of temporal variables.
- `uniq_avg`: Average uniqueness across all columns.
- `anchor_ratio`: Ratio of primary-key and foreign-key candidates to total columns. Represents relational connectivity.

Decision trees learn interpretable, rule-like decision boundaries from the extracted features. However, single decision trees are sensitive to variations in training data and may over-fit specific feature patterns. To address this limitation, we additionally employed random forest as an ensemble-based extension that improves generalisation through aggregation of multiple decision trees trained on randomised subsets of the data.

For semantic-based classification, we generated embedding representations from the textual metadata of the tables, which we used as inputs for classification. This enables the model to capture contextual and semantic relationships beyond manually engineered structural features. A table’s representation is built by turning its name and column names into embeddings, which are dense numerical vectors of floating-point numbers that a pre-trained language model produces for any input text. Embeddings are positioned in a high-dimensional space such that semantically similar texts land geometrically close together. For example, ‘patients’ and ‘admissions’ end up nearer each other than ‘patients’ and ‘chartevents’. We encoded the table name into one such vector, and grouped the column names into chunks, where each chunk is encoded into a separate vector. Finally, the table name vector and all chunk vectors are mean-pooled into a single fixed-size vector that represents the whole table’s representation.

We compared different configurations in the embedding approach, which were obtained by setting different embedding models or utilising different embedding-based classification strategies:

1. Building *per-class centroids* from labelled embeddings, where the centroid for a class is the mean of all same-class table vectors; building a *per-table embedding* for each table excluding that table; and finding the nearest per-class centroid by cosine similarity to each per-table embedding, which becomes the predicted class. This strategy is based on the assumption that tables of the same class should be closer to each other than to other classes.
2. Encoding the raw label strings "entity", "event", and "mixed" directly and finding the closest label string embedding for each table embedding based on cosine similarity, which becomes the predicted class. This is zero-shot by design and assumes that the table embedding can be classified by proximity to those label embeddings, which represent the semantic meaning of the class labels.

3.2.3 LLM-Based Method

In this method, we designed an LLM-based table classifier powered by LLM APIs obtained from the GroqCloud platform. The classifier works by prompting the LLM with a structured prompt consisting of a role, a task, definitions for entity, event, and mixed. The model produces a simple JSON structure, which consists of the predicted classification label and reasoning text.

We compare multiple configurations obtained by varying both the input representations provided to the LLM and the prompting strategies employed. Specifically, the method includes seven types of input representations and two prompt formulations. The input representations are as follows:

- Table Name (tbnm), where only the table name is passed.
- Column Names (c1nm), where the table name plus a flat list of column names are passed.
- Column Names with Samples (c1_samp), where table name and column names are combined with sampled rows.
- Schema (sch), table name with each column name and its inferred data type are passed.
- Schema with Samples (sch_samp), where the schema is combined with sampled rows.
- Entire Profile (prof), where the schema is extended with per-column uniqueness and null ratio statistics.
- Profile with Samples (prof_samp), where the entire statistical profile is combined with sampled rows.

The prompt strategies include a *zero-shot* setting, where only the role, input, and output specifications, task, and label definitions are provided, and a *few-shot + negative prompting* setting, which extends the zero-shot prompt by incorporating representative examples in each class and explicitly discouraging over-reliance on surface-level textual or structural cues. The prompts for the zero-shot and the few-shot strategy are mentioned in appendix B.1.

It should be noted that the classification and decomposition stages are designed in such a way that all tables get classified into entity, event or mixed tables before decomposition. An alternative design could have first separated mixed and non-mixed tables, followed by classifying non-mixed tables into entity and event categories, and then decomposing the mixed tables. Such a separation might have reduced the signal overlap introduced by mixed tables; however, its effectiveness cannot be established without implementation and empirical evaluation.

3.3 Multi-Entity Table Decomposition

The process of table classification produces predictions that are either entities, events, or mixed tables. In the dataset, mixed tables do not represent a single entity or event, but a combination of multiple entity and event-level information. This calls for designing a method that detects when a table contains multiple logical entities or events and proposes a decomposition. This involves detecting the boundaries between entities or events in the table, splitting the table into conceptually similar categories of columns, detecting nodes, and generating edges between nodes.

We explored three different decomposition methods, each using a different attribute to quantify or qualify column closeness.

- Functional Dependency-based column clustering
- Co-occurrence analysis using inclusion dependencies
- LLM-guided structural proposals

Regardless of the decomposition method used, the output is structured as a JSON object consisting of two main components: clusters and edges.

The `clusters` field is a list of cluster objects, where each cluster is assigned a unique, zero-indexed `cluster_id`. Each cluster contains a list of columns that belong to that cluster, including its identifier column. In addition, each cluster specifies a `candidate_primary_key`, which corresponds to the identifier column used to represent that cluster.

The `edges` field encodes relationships between clusters as a list of directed connections. Each edge specifies a `from_cluster` and a `to_cluster`, both referring to cluster identifiers. These edges represent inferred structural relationships between the corresponding clusters.

3.3.1 Functional Dependency-based column clustering

This method uses pre-discovered FDs from the hybrid functional dependency (HyFD) algorithm to build an affinity matrix between column pairs, then runs Louvain, a community detection algorithm, on the resulting matrix to detect clusters of columns.

The HyFD algorithm discovers all minimal, non-trivial functional dependencies from tables. FD discovery is computationally expensive because the search space for FDs increases exponentially with an increase in schema size. To manage this, the algorithm alternates between a column-efficient sampling phase and a row-efficient validation phase. The alternation process is dynamic and based on observations per comparison or valid FDs per validation.

We designed the method for building the affinity matrix by accumulating scores for each FD. The method is parametrised by an input weight configuration, which defines four numbers analogous to hyperparameters: w_{sym} , w_{asym} , w_{LHS_intra} , and w_{RHS_intra} . As the method encounters each FD " $X \rightarrow Y$ ", the following steps are ensured:

- Initially, a base importance weight(W) is computed using:

$$W = \frac{1}{|X|} \cdot \frac{1}{|Y|}.$$
- For each column pair x and y in the FD, i.e., for every $x \in X$ and $y \in Y$, the affinity matrix is incrementally updated as:

$$M[x][y] += W \cdot w_{inter}, \quad M[y][x] += W \cdot w_{inter},$$
where w_{inter} depends on whether the reverse dependency $Y \rightarrow X$ exists and uses symmetric (w_{sym}) or asymmetric (w_{asym}) weighting from the weight configuration.
- Additionally, for every intra-set column pairs, i.e., among column pairs within X and within Y , affinities are increased:

$$M[x_i][x_j] += W \cdot w_{LHS_intra} \text{ for all } x_i, x_j \in X,$$

$$M[y_i][y_j] += W \cdot w_{RHS_intra} \text{ for all } y_i, y_j \in Y.$$

The assumption is that columns inside X are related to each other because they jointly determine something, columns in Y are related to each other because they jointly depend on X , and columns between X and Y imply cross-group association.

After we construct the affinity matrix M , the method converts the matrix into a weighted graph, where each column becomes a node and each non-zero value $M[i][j]$ in the matrix becomes an edge weight between columns i and j . We use the Louvain algorithm to partition the graph into communities, where the objective is to create dense intra-group edges and sparse inter-group edges. Each community is treated as the decomposition cluster, and columns inside a community are treated as semantically related. Between every pair of clusters, a directed edge is drawn if there exists at least one asymmetric FD spanning the clusters. We draw the edge based on the strongest FD-induced inter-cluster connection. For each cluster, we select an identifier, either based on the end points of the strongest edge incident on it, or based on the column with the highest uniqueness within the cluster. The identifier selection method can also be specified in the input configuration. Finally, we label each directed edge with the identifier of the destination cluster and add that identifier as a column in the source cluster, creating a relationship analogous to a PK-FK relationship.

3.3.2 Co-occurrence analysis using inclusion dependencies

This method clusters columns using ‘anchors,’ which are either primary keys or foreign keys. We use these anchors as cluster identifiers and assign all the other columns to one of these anchors based on different ways of quantifying column closeness. The assumption behind the method is that the primary keys and foreign keys of the table define structural boundaries between entities and events, and the quantified closeness between two columns reflects their true relational proximity for the purpose of decomposing a table into substructures.

The closeness between an anchor and a non-anchor is quantified using three scoring variants. If a column is computed to be equally ‘close’ to two anchors, the tie is broken either by selecting the first column in the given order or by selecting the column with higher uniqueness. The scores are as follows:

- ‘*exact*’, which computes the fraction of anchor groups where the target column is perfectly consistent (single unique value).
- ‘*ratio*’, which evaluates how many unique values (*nunique*) the target column has, computes $1/nunique$, and aggregates across groups.
- ‘*weighted_entropy*’, which calculates the entropy of target column values within each anchor group, then computes a weighted aggregate of entropies using weights proportional to the number of observations in each group.

The anchors are the identifiers themselves, and an edge is created if there is a PK–FK mapping across two different clusters. We set the direction of the edge from the PK to the FK column and label the edge with the identifier of the FK cluster, which is also added as a column in the PK cluster.

3.3.3 LLM-guided structural proposals

We designed the LLM-based approach to decomposition similar to the LLM-based classification method, where there is a prompt consisting of a role and a task, along with instructions and rules to decompose, and input and output specifications. The method can accept different input representations, which are exactly the same as those for the classification. The output format is the same as those returned by the FD and IND-based methods, with the only difference being that the LLM also outputs a reasoning text along with the cluster and edge list. The core decomposition logic is as follows:

1. The LLM picks columns that are meaningful as identifiers for individual clusters. The number of identifiers picked defines the number of clusters.
2. Every non-identifier column is assigned to exactly one identifier-based cluster based on its meaning derived by the LLM.
3. The LLM generates edges between the appropriate clusters based on its notion of a semantic relationship. This includes the edge direction. The edge label, however, is deterministically picked, as the identifier of the source cluster, which is also copied into the destination cluster.

The decomposition process is executed iteratively, where each iteration includes the role, task, input representation, and a zero-indexed iteration number. The first iteration is for the initial decomposition, and subsequent iterations allow the LLM to improve upon the previous output. In each subsequent iteration, a deterministic validation is also given as input. The validation checks for: (i) completeness, ensuring that every column is assigned to exactly one cluster, (ii) uniqueness, preventing duplicate column assignments across clusters, and (iii) schema consistency, ensuring that all referenced columns and edges are valid with respect to the input table. The prompt for the first iteration and additional prompts added in the validation iterations are mentioned in appendix B.2.

Chapter 4

Experiments

We designed the experiments to align with the proposed transformation pipeline and to enable the systematic comparison of traditional automated methods and LLM-approaches across the different stages. Each stage is evaluated using task-appropriate metrics and analysis techniques, ranging from standard classification metrics for machine learning methods to a combination of qualitative and quantitative assessments for LLM methods.

The MIMIC-IV v3.1 dataset [Johnson et al., 2024] was selected as the baseline to develop and test methods for classifying tables into canonical types and decomposing mixed tables, and the e-ICU dataset v2.0 [Pollard et al., 2019] was used to test the robustness of our methods by applying the best-performing methods from MIMIC on this dataset.

4.1 Data

MIMIC-IV¹ is a dataset consisting of real, de-identified data of patients admitted to a tertiary academic medical centre in Boston, Massachusetts, USA. The dataset contains a comprehensive set of patient records, including lab measurements and vitals, prescriptions and medication, admission and discharge information, and much more. It is separated into 5 modules: *hosp*, which contains hospital-wide patient data; *icu*, which contains ICU-level data; *ed*, which is emergency department data; *cxr*, which contains data of chest radiographs; and *note*, which contains de-identified free-text clinical notes. In our experiments, we used MIMIC-IV v3.1, which contains *hosp* and *icu* information, as our baseline.

4.1.1 MIMIC-IV

We manually annotated the tables with the labels *entity*, *event*, and *mixed* as per the definitions chosen for each class. Under this annotation, the dataset consists of 8 entity tables, 21 event tables, and only 2 mixed tables. Specifically, across both *hosp* and *icu* modules, the tables *d_items*, *d_hcpcs*, *d_labitems*, *patients*, *d_icd_diagnoses*, *d_icd_procedures*, *provider*, and *caregiver* are entity tables, *icustays* and *admissions* are mixed tables, and *emar*, *ingredientevents*, *chartevents*, *services*, *pharmacy*, *drgcodes*, *emar_detail*, *diagnoses_icd*, *prescriptions*, *labevents*, *procedures_icd*, *procedureevents*, *microbiologyevents*, *outputevents*, *inpuvents*, *hpcsevents*, *datetimeevents*, *omr*, *transfers*, *poe*, and *poe_detail* are event tables.

As mentioned in subsection 3.2.2, we constructed an augmented dataset by creating new mixed tables by synthetically combining some entity and event tables, and creating new entity and event tables from splitting the mixed tables. Specifically, the following augmentation steps were taken:

- New mixed tables were created by approximately calculating statistics of the new tables from profiles, where uniqueness and null from the event table remain unchanged, and the uniqueness of the entity table is computed by dividing old uniqueness by the ratio of event to entity row count.

¹Information about MIMIC obtained from: <https://mimic.mit.edu/docs/IV/>

- The *patient* table was mixed as stated above with the event tables *hpcsevents*, *procedures_icd*, *labevents*, *services*, *microbiologyevents*, *transfers*, *omr*, *pharmacy*, *diagnoses_icd*, *poe*, *emar*, *prescriptions*)
- Tables *d_labitems*, *d_icd_diagnoses* and *d_icd_procedures* were mixed with *labevents*, *diagnoses_icd*, and *procedures_icd* respectively.
- In the *icu* module, the table *d_items* was split using the “linksto” column and joined with each *icu* event table.
- The mixed table *admissions* was split to create the *admissionevents* event and *admitted_patients* entity tables. Similarly, the mixed table *icustays* was split to create the *iculocations* entity and *icustays* event tables.
- We kept the original tables as part of the total dataset, which consists of base tables and augmented versions.

4.1.2 e-ICU

The e-ICU Collaborative Research Database² is a large database, consisting of data from critical care centres from multiple hospitals. The data is made available by Philips Healthcare in partnership with the MIT Laboratory for Computational Physiology. The dataset contains over 200,000 stay records of patients who were part of the Philips e-ICU program across ICUs in the United States, providing a large sample size for research studies. The tables are connected by unique identifiers, such as patient ICU stay and hospital IDs.

Similar to MIMIC, we manually annotated the e-ICU tables with the labels *entity*, *event* and *mixed* as per the definitions chosen for each class. It was observed that the e-ICU dataset consisted of a single entity table, 28 event tables, and 2 mixed tables. We annotated *hospital* as an entity table, *patient* and *apachePredVar* as mixed tables, and *admissionDrug*, *carePlanEOL*, *infusionDrug*, *nurseCare*, *treatment*, *admissionDx*, *carePlanGeneral*, *intakeOutput*, *nurseCharting*, *vitalAperiodic*, *allergy*, *carePlanGoal*, *lab*, *pastHistory*, *vitalPeriodic*, *apacheApsVar*, *carePlanInfectiousDisease*, *medication*, *apachePatientResult*, *customLab*, *microLab*, *physicalExam*, *diagnosis*, *note*, *respiratoryCare*, *carePlanCareProvider*, *nurseAssessment*, and *respiratoryCharting* as event tables.

We also constructed an augmented dataset for e-ICU by taking the following augmentation steps:

- We created mixed tables using a similar synthetic derivation of new table statistics.
- We used the *patient* table and derived new patient entity, patient hospital stay, and patient ICU stay tables.
- Similarly, we used the *apachePredVar* mixed table to create assessment event and comorbidity entity tables.
- We combined the new patient entity table with the other event tables to create mixed tables.
- We used the event tables such *lab*, *medication*, *admissionDrug*, *infusionDrug*, *diagnosis* and created respective entity tables.
- We kept the original tables as part of the total dataset, which consists of base tables and augmented versions.

4.2 Evaluation

4.2.1 Classification

For evaluating the classification approaches, we chose a set of metrics that consists of precision, recall, F1-score, and the classification report with a per-class breakdown.

²Information about e-ICU obtained from: <https://eicu.mit.edu/>

- Precision: Proportion of all positive predictions that are actually positive.

$$\text{Precision} = \frac{\text{Correctly Classified Actual Positives}}{\text{All Classified Positives}} = \frac{TP}{TP + FP} \quad (4.1)$$

- Recall or True Positive Rate: Proportion of all actual positives correctly classified as positive.

$$\text{Recall (or } TPR) = \frac{\text{Correctly Classified Actual Positives}}{\text{All Actual Positives}} = \frac{TP}{TP + FN} \quad (4.2)$$

- F1 score: Harmonic mean of precision and recall. F1 score balances the importance of precision and recall, and is preferable to accuracy for class-imbalanced datasets.

$$F1 = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \quad (4.3)$$

Note: TP = Number of True Positives, TN = Number of True Negatives, FP = Number of False Positives, FN = Number of False Negatives

4.2.2 Decomposition

We defined ground truth decompositions for all the mixed tables in the dataset and evaluated the predicted decomposition outputs across multiple aspects, comparing the outputs with the ground truth decompositions. The same evaluation was applied to all the decomposition approaches, which consist of the following:

- Cluster composition: The fraction of columns within each predicted cluster that actually belong together. A higher score indicates more cohesive predicted clusters.
- Cluster fragmentation: For each ground-truth cluster, the extent to which its columns are dispersed across multiple predicted clusters. A higher score indicates less fragmentation, meaning ground-truth clusters are more intact in the predicted decomposition.
- PK Metrics: For each predicted primary key, whether it exists in the ground-truth PK set, and the fraction of columns in its predicted cluster that share the same ground-truth cluster as that primary key. Higher scores indicate more accurate identifier selection and cluster coherence.
- Edge Metrics: For each predicted edge label, whether it exists in the ground truth edge set, and the similarity between the set of columns that are proximal to each ground truth label and the set of columns that are proximal to the equivalent column in the predicted decomposition. The word ‘proximal’ means that the column is in the same cluster as the reference column or in a cluster connected to the original cluster by a direct edge. Higher scores indicate more accurate relationship recovery.

4.3 Experiment methods

4.3.1 Profiling

To implement the profiling, we compared manual approaches with existing tools for profiling, as specified in table 4.1. For extracting the statistics of a single table, such as data types, uniqueness ratios, and null patterns, we selected an SQL-based profiling approach due to its ability to execute queries directly over raw CSV files without requiring explicit data ingestion into a separate database storage layer. SQL-based approaches leverage query optimisation and compiled execution plans, unlike Python-based profilers, e.g., pandas, which perform the same operations through slower, loop-based processing. This contributes to reduced memory overhead and execution time. In particular, the SQL-based implementation is carried out using DuckDB.

Table 4.1: Comparison of Manual vs SQL-based approaches for every table statistic

Statistic	Manual Approach	SQL Approach
Number of rows	For every row in CSV, increment a counter by 1	<code>SELECT COUNT(*) FROM _col</code>
Data type	Type of first non-null value in the column	<code>DESCRIBE SELECT * FROM read_csv_auto(...)</code>
Uniqueness ratio	Store every non-null value in a set and compute unique / total	<code>COUNT(DISTINCT column_name) / COUNT(*)</code>
Null ratio	Treat NULL as None or empty string and count manually during iteration	<code>COUNT(*) - COUNT(col)</code>

For selecting the right algorithm to extract INDs, we considered both FAIDA and SPIDER as candidate methods, with the same filtering step applied for both, as discussed in section 3.1. The filtering steps specifically chosen for MIMIC are:

- Single Column INDs only (number of columns in both LHS and RHS of the IND is 1)
- Cross-Table INDs only (table of the column in LHS and RHS is different)
- PK alignment (column in RHS is a PK)
- Type compatibility (type of the column in LHS and RHS is the same)
- A custom filter applied after observing the INDs (LHS column ends with `_id`)

In MIMIC, FAIDA is an approximate method that tends to produce a broader set of candidate dependencies, capturing a larger number of potential relationships between attributes. This leads to a higher recall of candidate dependencies. In contrast, SPIDER generated a more conservative set of dependencies due to its stricter exact containment verification, leading to fewer discovered relationships under identical filtering constraints. In SPIDER, several potentially meaningful dependencies were not surfaced under its stricter matching criteria. Hence, we selected FAIDA as the better IND discovery algorithm in the profiling step.

Similar to the MIMIC dataset, we also applied the structural profiling on the e-ICU dataset for extracting intra-table statistics and inclusion dependencies. Similar SQL-based approaches was used for computing intra-table statistics.

For extracting inclusion dependencies from e-ICU, SPIDER was determined to be the more suitable method than FAIDA, in contrast to the selection made for MIMIC. In general, the choice between FAIDA and SPIDER was determined based on analysis of schema complexity and required output. FAIDA was more suitable for datasets with moderate cross-table comparison spaces, where a higher recall of candidate dependencies was beneficial for profiling. In contrast, SPIDER was suitable for datasets with substantially more column comparisons, where memory-efficient execution becomes critical. This selection strategy explains the observed result where FAIDA was more suitable for MIMIC, while SPIDER proved more appropriate for the e-ICU dataset. Similar filtering steps were applied to the inclusion dependencies generated by FAIDA, with a different custom filter that checks for columns ending in ‘id’.

4.3.2 Classification

Rule-Based

We implemented and tested the rule-based method with two different rule sets. The rules, conditions, and threshold values in the first, old rule set were intuitively chosen, and the second, new rule set was generated by improving upon the first rule set based on performance and further intuition. The

Table 4.2: Rule-based classification: old rule set

Rule	Conditions	Confidence (ENTITY / EVENT / MIXED)
#1	has_pk = true fk_count $\leq$ 0	0.90 / 0.05 / 0.05
#2	timestamp_count $\geq$ 2 fk_count $\geq$ 1 col_count $\geq$ 8	0.05 / 0.15 / 0.80
#3	timestamp_count $\geq$ 2 fk_count $\geq$ 1	0.05 / 0.80 / 0.15

Table 4.3: Rule-based classification: refined rule set

Rule	Conditions	Confidence (ENTITY / EVENT / MIXED)
#1	has_pk = true fk_count $\leq$ 0	0.90 / 0.05 / 0.05
#2	timestamp_count $\geq$ 1 fk_count $\geq$ 1	0.05 / 0.75 / 0.15
#3	fk_count $\geq$ 2	0.05 / 0.50 / 0.90

performance of the rule-based method was compared between the predictions obtained using both rule sets. The tables 4.2 and 4.3 show the values for both the old and new rule sets, respectively.

ML Based: Decision Tree

For the decision tree method, we converted the input profiles to features, as discussed in subsection 3.2.2. Each row of the dataset used for the ML model contains a tuple of features for a single table, along with the annotated label. The dataset was split into an 80/20 stratified train-test partition and the experiment was run with random seed 42.

During training, we applied a 5-fold cross-validation procedure on the training set to assess model stability. The final model was trained using a maximum depth of 4 to control overfitting, with both entropy-based and gini-based splitting criteria evaluated across validation and final training stages.

ML Based: Random Forest

For the random forest method, we used the same input features as the ones in the decision tree method. The dataset was split into an 80/20 stratified train-test partition, and the experiment was run with random seed 42.

During training, we applied a 5-fold cross-validation procedure on the training set to assess model stability.

ML Based: Embeddings

We implemented and tested the embedding-based method with six different methods: three different embedding model and two different classification strategies as mentioned in subsection 3.2.2. The different models used were:

- BioBERT: a pre-trained biomedical language representation model
- all-MiniLM-L6-v2: a sentence-transformers model

Table 4.4: Rule-based method: Per-class and aggregated F1-score comparison

Rule Set	Dataset	Entity F1	Event F1	Mixed F1	Macro F1	Weighted F1
Old	Base Tables	0.59	0.67	0.00	0.42	0.60
Old	Base + Augmented	0.56	0.42	0.00	0.32	0.27
New	Base Tables	0.89	0.84	0.00	0.58	0.80
New	Base + Augmented	0.83	0.53	0.07	0.48	0.40

- SapBERT: a pretraining scheme that self-aligns the representation space of biomedical entities.

We set the chunk size to 10, for grouping column names into manageable segments before encoding, and used a maximum token length of 32 during tokenization.

LLM-Based

We implemented the LLM-based classifier according to subsection 3.2.3.

We used the Llama-4-Scout model, a 17-billion-parameter, instruction-tuned mixture-of-experts (MoE) large language model, obtained from the GroqCloud platform. A temperature of 0.8 was set, which produces a balance of coherent yet reasonably varied outputs, and the final classification label was determined using majority voting over three independent inference runs.

4.3.3 Decomposition

In the FD-Based method, we evaluated three weight configurations in our experiments. The configurations differ in how FDs are processed into affinity matrix weights. In the first configuration, equal weights are assigned to intra-LHS, intra-RHS, and inter-LHS-RHS column pairs, resulting in a balanced representation. In the second configuration, inter-LHS-RHS pairs are assigned a higher weight than intra-side pairs, promoting clustering driven primarily by connections across dependency sides. In the third configuration, intra-LHS relationships are weighted more strongly than intra-RHS relationships, biasing the affinity structure toward grouping columns that jointly act as determinants. Identifier selection is also a configurable component, based on the criteria given in subsection 3.3.1.

In the IND-Based method, we implemented and evaluated the method across three closeness metrics and two tie-breaker metrics.

In the LLM-based method, we evaluated the method across different input configurations, as specified in subsection 3.3.3, using a prompt that defines the role, task, and instructions.

4.4 Results

4.4.1 Classification

MIMIC-IV: Rule-Based

The results across all settings in the rule-based method were obtained as shown in Table 4.4.

An immediate observation here is that Macro, or unweighted F1 scores, are worse than weighted F1 when it comes to the base case, while slightly better than weighted in the base + augmented case. This is because the mixed performance is hidden by its small class size, leading to higher weighted scores, while in the augmented case, the mixed still performs poorly, but now has a much higher class size, leading to

Table 4.5: Decision tree method: Per-class and aggregated F1-score comparison

Criteria	Dataset	Entity F1	Event F1	Mixed F1	Macro F1	Weighted F1
Gini	Base Tables	0.67	0.67	0.00	0.44	0.67
Gini	Base + Augmented	0.67	0.73	0.80	0.73	0.75
Entropy	Base Tables	0.67	0.67	0.00	0.44	0.67
Entropy	Base + Augmented	0.67	0.73	0.80	0.73	0.75

a drop in weighted average. The main observations are that the rule-based method did not specifically benefit from augmentation, likely because the method does not allow the fixed set of rules to tune itself to better class distribution. The mixed performance is poor across all cases, pointing to insufficient separability of mixed tables by the current rules, but the new rules allow for better separability between entity and event classes. In general, the definition of rules require very careful manual definition of the rules, conditions, and thresholds, making it less scalable and less generalizable compared to data-driven approaches.

MIMIC-IV: Decision Tree

Performance was assessed on a held-out test set using previously discussed classification metrics, as shown in Table 4.5.

Here it is seen that macro is always less than weighted F1, unlike the previous case, likely attributed to the observation that mixed increases in class size but also increases in performance. Decision trees mathematically learn decision boundaries themselves, compared to the manually set conditions and thresholds in the rule-based method. This leads to better class separability and performance compared to the results of the rule-based method. The augmentation, apart from improving the number of training samples, also improved the diversity of structural representations within each class. There was no difference observed between the Gini and entropy criteria, likely because both impurity measures produced comparable partitioning behaviour within the profiling feature space.

The improvement in performance from the rule-based method to the decision tree indicates that the classes contain an underlying separability that can be captured with automatically derived decision boundaries, as opposed to manually defined conditions. Augmentation introduces additional structural diversity and improves separability, but the model can only exploit a finite number of partition regions. Once the most discriminative feature splits are exhausted, it is observed that performance saturates. This saturation motivates the exploration of ensemble-based approaches. Random forests, for e.g., relax the constraints imposed by a stand-alone decision tree and can better exploit structural variability introduced by the data.

MIMIC-IV: Random Forest

Performance was assessed on a held-out test set using previously discussed classification metrics, as shown in Table 4.6.

The additional ensembling over multiple decision trees introduced by the random forest method reduces model variance via averaging. Thus, it adds diversity in decision boundaries. The base dataset still performs poorly, likely attributed to the reduced amount of mixed samples. Augmentation adds structural diversity similar to the decision tree method. Furthermore, in random forests, the ensemble framework can exploit this added diversity through multiple decorrelated decision boundaries. This is evident in the performance jump from the base dataset to the augmented version, where augmentation

Table 4.6: Random Forest method: Per-class and aggregated F1-score comparison

Dataset	Entity F1	Event F1	Mixed F1	Macro F1	Weighted F1
Base Tables	0.67	0.80	0.00	0.49	0.76
Base + Augmented	1.00	0.91	0.89	0.93	0.92

Table 4.7: Embedding-based method: Per-class and aggregated F1-score comparison

Model	Strategy	Dataset	Entity F1	Event F1	Mixed F1	Macro F1	Weighted F1
BioBERT	Centroid	Base Tables	0.67	0.81	0.00	0.49	0.72
		Base + Augmented	0.53	0.73	0.64	0.63	0.66
	Label	Base Tables	0.20	0.62	0.20	0.34	0.48
		Base + Augmented	0.15	0.43	0.18	0.25	0.28
all-MiniLM-L6-v2	Centroid	Base Tables	0.57	0.84	0.00	0.47	0.72
		Base + Augmented	0.56	0.61	0.54	0.57	0.57
	Label	Base Tables	0.57	0.53	0.00	0.37	0.51
		Base + Augmented	0.40	0.44	0.08	0.31	0.28
SapBERT	Centroid	Base Tables	0.75	0.84	0.00	0.53	0.76
		Base + Augmented	0.57	0.63	0.60	0.60	0.61
	Label	Base Tables	0.61	0.70	0.00	0.44	0.63
		Base + Augmented	0.41	0.61	0.00	0.34	0.32

is observed to have a greater impact than in the case of the decision tree, and the separability between classes is greater as well.

MIMIC-IV: Embedding-Based

Performance was assessed on the full dataset using previously discussed classification metrics, as shown in Table 4.7.

From the results, we can observe that the centroid-based strategy is higher than the label strategy across all settings. In the centroid method, the base dataset shows slightly higher metrics than the augmented configuration; while macro-F1 increases, weighted-F1 decreases. Augmentation introduces additional samples, which shifts the class centroids toward a more balanced position relative to the three classes. This likely explains the reduced performance for entity and event, while mixed improved. In contrast, the label method shows significantly worse metrics under augmentation. In this case, augmentation does not provide a meaningful benefit because there is no group-level learning. Instead, the method repeatedly performs independent comparisons, leading to a higher accumulation of misclassified examples as more data is introduced.

Even in the base setting, metrics remain weak, likely because the embeddings of the class labels “entity”, “event”, and “mixed” do not carry any significant closeness in meaning with respect to the table inputs. The centroid-based method, which attempts to cluster semantically related tables closer to each other, is better aligned with the underlying structure of the task.

Additionally, SapBERT is shown to output slightly better metrics than the other embedding models. This is likely due to its stronger alignment with biomedical entity representations, making its embedding space more suitable for this domain. This improved domain alignment may also partially explain why the performance gap between centroid-based and label-based methods is slightly reduced for SapBERT compared to the other embedding models.

Table 4.8: LLM-based method: Per-class and aggregated F1-score comparison - Zero-Shot System Prompt with definitions

Input	Entity F1	Event F1	Mixed F1	Macro F1	Weighted F1
tbnm	0.62	0.56	0.00	0.39	0.54
clnm	0.80	0.84	0.00	0.55	0.78
sch	0.89	0.83	0.50	0.74	0.83
prof	0.80	0.60	0.33	0.58	0.63
cl_samp	0.80	0.74	0.00	0.51	0.71
sch_samp	0.76	0.73	0.25	0.58	0.71
prof_samp	0.89	0.65	0.31	0.61	0.69

Table 4.9: LLM-based method: Per-class and aggregated F1-score comparison - Few-Shot System Prompt with definitions and restriction

Input	Entity F1	Event F1	Mixed F1	Macro F1	Weighted F1
tbnm	0.84	0.89	0.80	0.85	0.88
clnm	0.94	0.90	0.40	0.75	0.88
sch	0.94	0.95	0.80	0.90	0.94
prof	0.89	0.83	0.50	0.74	0.83
cl_samp	0.89	0.92	0.80	0.87	0.91
sch_samp	0.89	0.92	0.80	0.87	0.91
prof_samp	0.89	0.86	0.57	0.78	0.85

The embedding-based methods appear to primarily capture structural similarity in table schemas rather than the functional semantics required for classification. As reflected in the predictions, similarity is largely driven by repeated and high-frequency identifiers such as `subject_id`, `hadm_id`, `itemid`, and `icd_code`. The target labels entity, event, and mixed are defined by the functional role of each table within the database. Hence, there is a mismatch between the output of the embedding model and functional class definitions, which leads to weak separability. This is especially evident in mixed, where even the most aligned embedding model shows limited performance.

MIMIC-IV: LLM-based

Performance was assessed on the base dataset, using previously discussed classification metrics, as shown in Tables 4.8 and 4.9.

In the zero-shot results in Table 4.8, the metrics are observed to consistently improve as progressively richer input representations are provided, particularly when moving from table names to schema-level information. However, this trend is not consistently maintained in profile-based inputs, where slight performance degradation is observed. An explanation is that numerical attributes such as uniqueness and null ratios, as well as ID-like columns, introduce low-semantic signals that disproportionately influence the model’s reasoning process. Similarly, the inclusion of raw sample values introduces instance-level numerical data, which shifts attention away from the semantic meaning of the table. These signals likely dilute the influence of task-relevant structural features and weaken alignment with the functional decision boundary required to distinguish entity, event, and mixed tables.

We observe a very different phenomenon in the few-shot case. The inclusion of specific examples and explicit constraints enables the model to more reliably distinguish between tables that “describe a concept” as entity-like and those that “record an occurrence of that concept” as event-like, as illustrated by cases such as `procedures_icd` versus `d_icd_procedures`. The addition of samples is not observed to

Table 4.10: Random Forest method on e-ICU dataset: Per-class and aggregated F1-score comparison

Dataset	Entity F1	Event F1	Mixed F1	Macro F1	Weighted F1
Base Tables	–	0.91	0.00	0.45	0.91
Base + Augmented	1.00	0.77	0.73	0.83	0.78

significantly improve performance, indicating that the few-shot examples and imposed restrictions were the main drivers of the more refined reasoning.

The LLM-based method is able to understand semantic meaning within tables, relying on pre-trained external knowledge and prompt-driven reasoning instead of manually engineered structural features. The prompts explicitly encode definitions of entity, event, and mixed table types. However, LLM behaviour remains inherently prompt-sensitive and less deterministic compared to traditional feature-driven approaches, primarily driven by semantic alignment between textual representations and external knowledge rather than explicit structural signals in the data. It also shows limited benefit, and in some cases degradation, when incorporating numeric profiling features or additional samples. In contrast, the random forest model operates exclusively over manually engineered features, such as timestamp density, uniqueness ratios, and relational keys, which directly encode table-level structural behaviour. These features are observed to induce stable decision boundaries, separating entity and event tables, while mixed tables occupy intermediate regions in the feature space. Since the downstream objective of the pipeline is a canonical graph representation, the random forest model was selected as the preferred learning approach, as the experiments show that its decision boundaries correspond directly to structural properties associated with entity, event, and mixed table behaviour relevant to graph representation.

e-ICU: Random Forest

We ran the chosen method, supervised ML with random forest, on the e-ICU dataset and used a similar evaluation.

We used the feature extraction module to construct the feature dataset, and implemented the random forest method using the same configuration as that applied to the MIMIC dataset. Performance was assessed on a held-out test set using previously discussed classification metrics, as shown in Table 4.10.

In the base dataset, the entity class is effectively omitted in the holdout split as there was only a single entity in the dataset. This resulted in misleadingly high performance, where nearly all samples are classified as events. The addition of augmentation enables coverage across all classes and hence a more meaningful evaluation, even though the overall F1 score is lower. The entity class in e-ICU is learned with similar clarity as in MIMIC due to similarly strong structural features; however, there is a noticeably higher degree of overlap between event and mixed categories.

In MIMIC, the mixed class tends to emerge from a hybrid of weaker structural signals such as lower uniqueness, moderate timestamp density, and intermediate row counts, while event is more strongly characterized by higher row counts and lower identifier concentration, in addition to strong contributions from `anchor_ratio` and `ts_ratio`, which correspond to cross-table referencing and temporal density respectively. In e-ICU, the feature space exhibits greater overlap between event and mixed categories, which likely explains the reduced performance in those classes while entity performance remains consistently high.

4.4.2 Decomposition

The full evaluation tables are provided in appendix A.

MIMIC-IV: FD-based

The FD-based method performs moderately well on column clustering metrics. In the equal-weight intra configuration, patient and admission columns are frequently mixed into the same clusters. Since intra-LHS, intra-RHS, and inter-LHS-RHS relationships are weighted equally, there is limited clear separation between the columns. In the configuration with higher LHS weighting, the objective is to group columns that jointly act as determinants. In practice, this leads to clusters where a greater number of columns are absorbed into dominant groups. This effect is likely driven by the presence of redundant FDs, which may repeat the same structural relationships. In contrast, the inter-dominant configuration yields higher scores on pure clustering metrics. This suggests that the connection across the left-hand and right-hand sides of a dependency appears to provide a more reliable basis for grouping than shared participation within a single side of the dependency.

The quality of edge labels and identifier selection remains limited. Both the uniqueness-based and edge-derived identifier selection methods do not consistently produce semantically stable cluster representatives. Directly using column uniqueness as a proxy for identity is also suboptimal since it is computed from a mixed profile. The uniqueness-based and edge-anchor methods differ only in their identifier selection strategy. The uniqueness-based approach yields more reliable identifiers, likely because high column uniqueness better approximates the notion of a stable cluster identifier than selecting the endpoint of the strongest incident edge. The latter is more susceptible to distortion from structurally non-informative FDs, which can bias edge strength. Notably, only the inter-LHS-RHS configuration produced a meaningful edge corresponding to the patient-admission relationship.

The *icustays* table likely yielded cleaner FDs overall, which provides a plausible explanation for the consistent separation of the *careunit* attributes into their cluster. Due to the relatively simple structure of the table, no meaningful variation in clustering behavior is observed between different configurations.

MIMIC-IV: IND-based

There is no observed difference between the tie-breaker methods, likely because the first identifier in the ordering is also the one with the most uniqueness, or that ties are rare themselves. The ratio and exact closeness metrics yield identical performance, which is also reflected in the resulting decompositions. Under the exact matching criterion, *hadm_id* is highly likely to produce unique values, leading to predominantly singleton groups, consequently, more consistent groupings. Although the ratio-based variant relaxes this strict matching behaviour, it does not materially affect the outcome, as singleton groups continue to dominate.

The weighted entropy variant shows lower metrics, despite producing more structurally meaningful decompositions in which a greater proportion of patient-related columns are grouped within the same cluster. In comparison to the patient identifier, *hadm_id* is more likely to produce consistent groupings. However, the criterion also penalizes smaller groups, which leads to *hadm_id* being less frequently chosen over patient and admit provider attributes. Despite these more meaningful groupings, the overall evaluation metrics are low because the evaluation also depends on the amount of optimal column pairings across all clusters, and *hadm_id* is not grouped with the right columns under the weighted entropy criterion.

Similar to the observation in the FD results, the *icustays* table results in the exact same metrics across all configurations. Moreover, in the actual decomposition, all columns are grouped together into a single cluster. This behaviour arises because clusters are determined by the presence of primary or foreign keys, and *careunit* does not contain a distinct identifier that would allow it to be recognized as a separate column category. This observation is not fully captured by the evaluation and requires manual inspection of the decompositions.

The edge and identifier selection performs very well for the *admissions* table, as the relevant relationships are directly reflected through the primary and foreign keys used by the IND method. In contrast, performance is lower for *icustays*, which follows from the clustering-related issues observed.

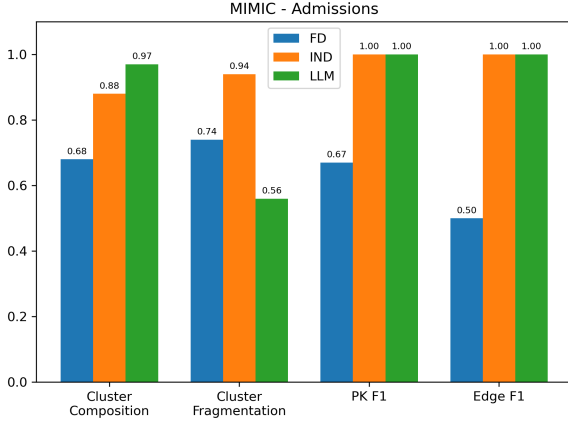


Figure 4.1: MIMIC Dataset Admissions Table: Comparison plot across decomposition methods

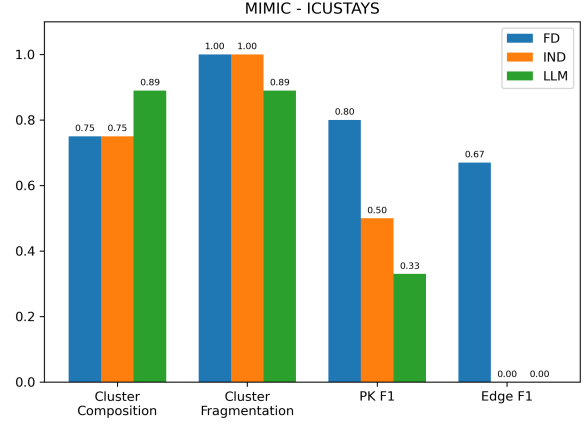


Figure 4.2: MIMIC Dataset ICUSTAYS Table: Comparison plot across decomposition methods

MIMIC-IV: LLM-based

The table input performs poorly as expected, since the columns are not provided in the input, and so the model creates its own decomposition. The remaining methods show comparatively strong performance on the admissions table. The columns-only input performs exceptionally well, as it correctly captures the semantic meaning of patient demographic attributes and their direct relationship to `subject_id`, while grouping the remaining attributes accordingly. Column-level information is observed to be a highly relevant and sufficient representation for decomposing the table. While richer inputs provide additional context, they also introduce noise that can cause the model to overemphasize details such as numbers and statistics. A similar effect was observed in the LLM-based classification experiments. The sample rows likely introduced the same type of confusion in the model’s reasoning process.

Similar to the other experiments, the *icustays* table shows consistent metrics across all the configurations. However, the metrics are slightly lower, as the decomposition consistently separates `subject_id` and `hadm_id` from the remaining columns. While this separation is semantically meaningful, it deviates from the ground truth structure. The method also struggles to separate the careunit attributes into distinct clusters, which is a more important deviation from the ground truth.

The LLM-based experiments produce insightful decompositions, as LLM models are generally able to understand the semantic meaning of the columns and group them accordingly. This behaviour is not reliably replicated by any of the statistical methods evaluated in the experiments. The models also find the identifiers based on their semantic roles without requiring additional profiling information, which reflects the advantage of LLM knowledge in this scenario. A comparison of decomposition methods across the MIMIC tables is presented in figures 4.1 and 4.2.

e-ICU: LLM-based

We evaluated the selected decomposition method, which was LLM-based, on the e-ICU dataset using a similar evaluation setup. The e-ICU patient table is comparable to the MIMIC admissions table, containing patient-level, hospital-level, and ICU-unit-level information. The schema is relatively simple, with hospital-level attributes typically prefixed with `hospital` and ICU-unit attributes prefixed with `unit`, which helps in straightforward clustering. Both column-only and schema-based inputs perform well in this setting, whereas profile-based inputs again tend to introduce noise and reduce performance. Identifier and edge selection in the patient table is similarly strong, consistent with the behaviour observed on the MIMIC dataset.

A surprising result is observed for the *apachePredVar* table, where the column-only input performs poorly compared to the schema and profile-based inputs. The structure of the *apachePredVar* table itself

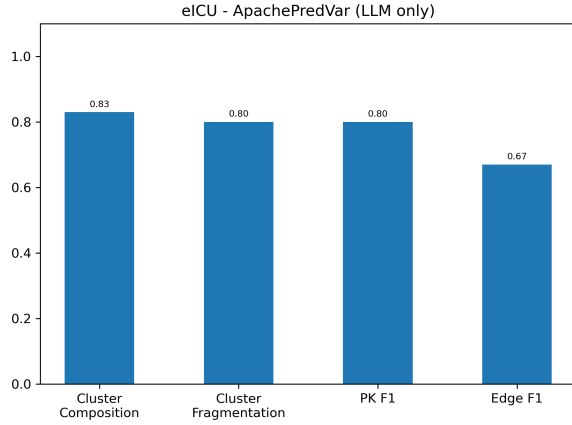


Figure 4.3: e-ICU Dataset ApachePredVar Table: Comparison plot across decomposition methods

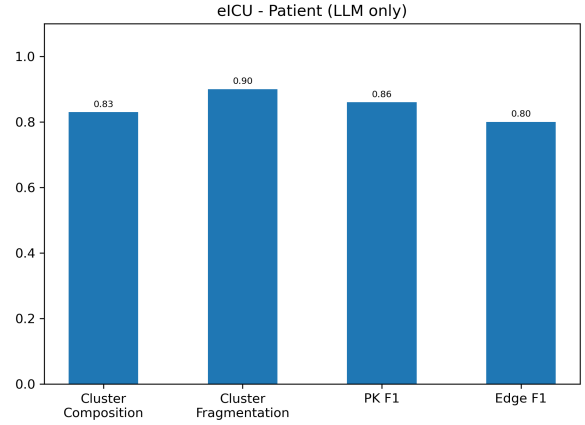


Figure 4.4: e-ICU Dataset Patient Table: Comparison plot across decomposition methods

is complex, as it contains patient context, clinical conditions, ICU admission context, measurements, and comorbidities. The ground truth decomposition includes a bridge cluster connecting multiple other clusters, resulting in a non-trivial decomposition goal. The sample-augmented column input performs particularly well in this setting, as it successfully identifies and separates the bridge cluster in addition to the other groupings.

The inclusion of samples leads to varied results, where the sample-augmented version achieves comparable or, in some cases, slightly better performance than the base model. This may suggest that the samples improve the model’s understanding of column semantics; however, this conclusion is not fully reliable based on the observed results. LLM performance across the e-ICU tables is presented in figures 4.3 and 4.4. In the figures, CC, CF, PF, and EF denote cluster composition, cluster fragmentation, PK F1 and Edge F1 respectively.

The LLM-based experiments demonstrate the model’s ability to decompose tables into meaningful clusters and represent their structure based primarily on semantic understanding. This indicates that the capabilities of LLMs are well aligned with the complex nature of the decomposition task and that semantic understanding is a necessary prerequisite for effective performance.

Chapter 5

Discussion

This chapter discusses the results of the experiments in relation to the three research questions posed in this thesis. The first question concerns how EHR tables can be classified into canonical types, namely entities and events. The second examines how mixed tables, containing both entity- and event-level information, can be identified and decomposed into meaningful substructures. The third evaluates how traditional automated methods and LLM-based approaches compare across the different stages of the pipeline, spanning profiling, classification, and decomposition.

5.1 RQ1: Entity-Event Classification

Identifying the functional roles of different elements of a knowledge graph is beneficial for organising and linking heterogeneous data sources. In EHRs, clinical records are typically structured to show a comprehensive view of a patient, encompassing facts such as biodata and prescriptions, observations such as vital signs and laboratory tests, interpretations such as diagnoses, plans or actions such as medications, and outcomes such as different procedures. From a data modelling and graph representation perspective, these categories can be broadly abstracted into entities and events. Our experiments show that these functional definitions of entities and events form highly distinguishable categories and can be used to classify multiple tables that share similar structural signatures. In particular, statistical approaches that utilise profiling information enable the extraction of complex structural interpretations of tables and achieve strong separability between entity and event types.

We further define mixed tables based on our observation, referring to tables that contain both entity- and event-level information. Such structures likely arise from denormalisation practices in EHR systems, often introduced for reasons such as billing or administrative efficiency. Furthermore, our results indicate that mixedness is a fundamentally more challenging category to identify. Across all our experiments, performance on mixed tables is consistently lower than on more structurally homogeneous entity and event tables. This difficulty can be attributed to the inherently relative definition of mixed tables, which is derived from the concepts of entity and event themselves. As a result, mixed tables may be interpreted either as tables that do not clearly belong to a single category, or as tables containing a combination of entity- and event-like columns. This ambiguity reduces classification stability and leads to increased sensitivity to feature representation and modelling assumptions. The presence of mixed tables can improve overall class coverage by introducing additional intermediate cases, which may support separability between the more clearly defined entity and event categories. Furthermore, mixed tables in real-world datasets may represent multiple heterogeneous subtypes, differing in both statistical features and specific entity–event compositions. These variations may benefit from additional semantic guidance, as observed in LLM-based approaches, which can partially mitigate structural ambiguity through contextual interpretation.

5.2 RQ2: Multi-Entity Table Decomposition

Following the identification of mixed tables, the decomposition task further highlights the need to transform mixed tables containing multiple information groups into graph-based representations of self-contained semantic units, with explicit relationships between them. This process is closely analogous to normalisation in relational database design. Achieving this requires recovering substructures through clustering columns, selecting appropriate identifiers, and establishing relationships between resulting groups. Our formulation of the tasks is explicitly grounded in the input data and provided annotations, rather than external knowledge bases or ontologies. While the LLM-based approach incorporates external semantic knowledge during grouping, its outputs are still evaluated deterministically against the ground truth.

Our results indicate that purely structural methods often provide incomplete approximations of the true decomposition. FD-based approaches can produce reasonable column groupings when inter-FD pairs are weighed higher; however, they struggle with identifier selection and relationship discovery due to their limited ability to distinguish between structurally similar but semantically distinct attributes. Similarly, IND-based methods are effective at selecting cluster identifiers through primary and foreign key relationships, often yielding identifier assignments equal or close to the ground truth, but they fail in cases where entity or event groups are not explicitly represented through key constraints. In the case of the IND-based method, certain evaluation metrics are partially misleading, as they reward larger correctly grouped clusters, leading to overlooking smaller but important clusters being incorrectly grouped.

This is where LLM-based methods demonstrate an advantage in that both column grouping and identifier selection are guided by semantic relatedness rather than purely structural features. This enables more flexible detection of relationships between attributes, particularly in cases where structural signals are weak or absent. Although ground truth decompositions may themselves be somewhat subjective, since mixed tables can have multiple valid structural decompositions, our experiments show that traditional methods tend to exhibit consistent deviations along specific dimensions, whereas LLM-based methods are more variable but better at capturing relationships between similar columns.

Overall, the combination of (i) deviation of structural abstractions from actual groupings, (ii) the semantically informed nature of ground truth construction, and (iii) the superior performance of LLM-based methods across multiple evaluation dimensions provides strong evidence that decomposition greatly benefits from the semantic understanding of column relationships.

5.3 RQ3: Comparison of Traditional and LLM-based Approaches

When examining the behaviour of traditional automated methods, it is evident that they are designed to exploit specific deterministic signals that are assumed to reflect relevant characteristics of the task. Consequently, the performance of each method provides an indication of how closely these assumed signals align with the actual underlying structure of the task, as this alignment is not always guaranteed. In the profiling stage of the pipeline, it is assumed that a fixed set of statistical signals can adequately represent the overall structure of a table. While it is not possible to assess the absolute correspondence between the profiling abstraction and the true underlying table structure, the profiling methods nevertheless produce consistent and reproducible representations of table characteristics. By design, the profiling stage relies purely on structural features.

We observe that traditional methods trained on these statistical profiles achieve strong performance due to the separability of entity and event classes through structural signals and the informative nature of features used to capture relevant distinctions. This suggests that carefully designed features can effectively encode aspects of the task definition, particularly for transforming tables into entity- and event-like graph representations. The reduced performance observed in the mixed category also highlights the difficulty of expressing mixedness through purely structural signals, as reflected in our experiments. In such cases, LLM-based reasoning may offer advantages by supporting the identification

of ambiguous or hybrid table structures that do not conform to pure categories.

In the decomposition stage, the consistently deviating performance of non-LLM methods indicates that decomposition is a complex, multi-step task for which designing purely traditional automated approaches that align closely with the underlying task definition is significantly more difficult. In contrast, semantic understanding appears to be both necessary and beneficial for achieving closer alignment with the intended decomposition structure.

Taken together, our results suggest that as we progress across profiling, classification, and decomposition, we are able to identify an increasing task complexity, which is associated with decreasing alignment between traditional methods and the underlying task definition, and a corresponding increase in the benefits of incorporating semantic reasoning through LLM-based approaches.

Chapter 6

Conclusion

Through this thesis, we have developed a proof-of-concept pipeline that transforms EHR tables into graph-based intermediate representations through profiling, classification, and decomposition. The resulting canonical graph elements can be stored within a graph database and integrated with graph structures derived from other data sources, such as ECG signals, clinical notes, and wearable devices. This unified representation enables the linking of heterogeneous clinical sources, supports the enrichment of relationships through external knowledge and LLM-based reasoning, as well as the inference of new connections via predictive modelling pipelines.

The profiling stage provided a common structural basis for both traditional and LLM-based approaches, where the appropriate input was selected from the profile and provided to the corresponding method. Rule-based, supervised ML, and LLM approaches were evaluated for assigning tables to entity, event, and mixed categories. Comparing these methods revealed which table characteristics are useful for graph-role identification. The experiments demonstrated how EHR tables can be classified into canonical types. FD-, IND-, and LLM-based decomposition methods were used to decompose mixed tables into meaningful substructures. Evaluating clustering, identifier selection, and relationship recovery allowed analysis of different decomposition strategies. The experiments demonstrated how mixed tables can be decomposed using the developed approaches. Applying traditional and LLM-based approaches across profiling, classification, and decomposition enabled comparison of their suitability at different stages. The experiments helped identify where structural signals are sufficient and where semantic reasoning becomes beneficial.

Specifically, our results demonstrate that profiling is, by design, a structure-oriented stage; that entity–event classification is largely achievable through structural signals; that semantic-driven approaches can be beneficial for identifying mixed tables; and that semantic signals can become necessary for decomposing mixed tables into meaningful substructures.

Assumptions and Limitations

This thesis is built on a number of limitations and assumptions, which have influenced aspects of the experimental design and evaluation.

- The evaluation of decomposition could have been extended beyond a strict comparison to a manually defined ground truth, as the assumption of a single correct decomposition may be overly restrictive. This limitation motivated the use of numerical metrics as indicative rather than definitive evidence, complemented by direct inspection of individual decomposition outputs.
- In addition, non-deterministic evaluation strategies, such as LLM-as-a-judge approaches (e.g., G-EVAL), could have been used to evaluate the quality of decomposition outputs.
- Furthermore, LLM-generated reasoning outputs should be interpreted with caution, as they mostly represent post-hoc rationalisations rather than faithful representations of the model’s internal decision-making process.

- Finally, the pipeline could be extended into a more tightly integrated, end-to-end system with explicit error propagation and feedback across stages.

Bibliography

- Francisco Abad-Navarro and Catalina Martinez-Costa. A knowledge graph-based data harmonization framework for secondary data reuse. *Computer Methods and Programs in Biomedicine*, 243:107918, 2024.
- Jana Bauckmann, Ulf Leser, Felix Naumann, and Véronique Tietz. Efficiently detecting inclusion dependencies. In *2007 IEEE 23rd international conference on data engineering*, pages 1448–1450. IEEE, 2007.
- Emily M Bender and Alexander Koller. Climbing towards nlu: On meaning, form, and understanding in the age of data. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 5185–5198, 2020.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Xin Luna Dong. Generations of knowledge graphs: The crazy ideas and the business impact. *Proc. VLDB Endow.*, 16(12):4130–4137, August 2023. ISSN 2150-8097.
- Lisa Ehrlinger and Wolfram Wöß. Towards a definition of knowledge graphs. *SEMANTiCS (Posters, Demos, SuCCESS)*, 48(1-4):2, 2016.
- R Scott Evans. Electronic health records: then, now, and in the future. *Yearbook of medical informatics*, 25(S 01):S48–S61, 2016.
- JaWanna Henry, Yuriy Pylypchuk, Talisha Searcy, and Vaishali Patel. Adoption of electronic health record systems among us non-federal acute care hospitals: 2008-2015. *ASTP health IT data brief [internet]*, 2016.
- Alistair Johnson, Lucas Bulgarelli, Tom Pollard, Brian Gow, Benjamin Moody, Steven Horng, Leo Anthony Celi, and Roger Mark. MIMIC-IV. *PhysioNet*, October 2024. Version 3.1.
- Ellen Kim, Samuel M Rubinstein, Kevin T Nead, Andrzej P Wojcieszynski, Peter E Gabriel, and Jeremy L Warner. The evolving use of electronic health records (ehr) for research. In *Seminars in radiation oncology*, volume 29, pages 354–361. Elsevier, 2019.
- Sebastian Kruse, Thorsten Papenbrock, Christian Dullweber, Moritz Finke, Manuel Hegner, Martin Zabel, Christian Zollner, and Felix Naumann. Fast approximate discovery of inclusion dependencies. In *Datenbanksysteme für Business, Technologie und Web (BTW 2017)*, pages 207–226. Gesellschaft für Informatik, Bonn, 2017.
- Jiayi Li, Daniel Garijo, and María Poveda-Villalón. Large language models for ontology engineering: a systematic literature review. *Semantic Web Journal*, 20(x):1–45, 2025.
- Prashanti Manda. Large language models in bio-ontology research: A review. *Bioengineering*, 12(11):1260, 2025.

- Nandana Mihindukulasooriya, Sanju Tiwari, Carlos F Enguix, and Kusum Lata. Text2kgbench: A benchmark for ontology-driven knowledge graph generation from text. In *International semantic web conference*, pages 247–265. Springer, 2023.
- Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology*, 16(5):1–72, 2025.
- Steve G Peters and James D Buntrock. Big data and the electronic health record. *The Journal of ambulatory care management*, 37(3):206–210, 2014.
- Tom Pollard, Alistair Johnson, Jesse Raffa, Leo Anthony Celi, Omar Badawi, and Roger Mark. eICU Collaborative Research Database. *PhysioNet*, April 2019. Version 2.0.
- Zhangcheng Qiang, Weiqing Wang, and Kerry Taylor. Agent-om: Leveraging llm agents for ontology matching. *Proc. VLDB Endow.*, 18(3):516–529, November 2024. ISSN 2150-8097.
- Bing Tong, Yan Zhou, Chen Zhang, Jianheng Tang, Jia Li, and Lei Chen. Galaxyweaver: Autonomous table-to-graph conversion and schema optimization with large language models. *Proceedings of the VLDB Endowment*, 18(12):5100–5112, 2025.
- Rui Yang, Ting Fang Tan, Wei Lu, Arun James Thirunavukarasu, Daniel Shu Wei Ting, and Nan Liu. Large language models in health care: Development, applications, and challenges. *Health Care Science*, 2(4):255–263, 2023.

Appendix A

Full Decomposition Evaluations

The following list shows the expansion of each metric used in the below tables A.1, A.2, A.3, and A.4. Further explanations are given in subsection 4.2.2.

- **CC**: Cluster Composition
- **CF**: Cluster Fragmentation
- **PP**: PK Precision
- **PR**: PK Recall
- **PF**: PK F1
- **PCC**: PK Cluster Coherence
- **EP**: Edge Label Precision
- **ER**: Edge Label Recall
- **EF**: Edge Label F1
- **ECC**: Edge Label Proximity

Table A.1: FD Method on MIMIC: Full Decomposition Evaluation

Case	CC	CF	PP	PR	PF	PCC	EP	ER	EF	ECC
adm_equal_intra_edge	0.63	0.80	0.50	0.33	0.40	0.38	0.00	0.00	0.00	0.78
adm_equal_intra_uniq	0.63	0.80	0.50	0.33	0.40	0.38	0.00	0.00	0.00	0.78
adm_inter_dominant_edge	0.68	0.74	0.33	0.33	0.33	0.49	0.00	0.00	0.00	0.78
adm_inter_dominant_uniq	0.68	0.74	0.67	0.67	0.67	0.49	0.50	0.50	0.50	0.78
adm_intra_lhs_high_edge	0.62	0.76	0.50	0.33	0.40	0.25	0.00	0.00	0.00	0.78
adm_intra_lhs_high_uniq	0.62	0.76	1.00	0.67	0.80	0.25	1.00	0.50	0.67	0.78
icu_equal_intra_edge	0.75	1.00	1.00	0.67	0.80	0.67	1.00	0.50	0.67	0.88
icu_equal_intra_uniq	0.75	1.00	1.00	0.67	0.80	0.67	1.00	0.50	0.67	0.88
icu_inter_dominant_edge	0.75	1.00	1.00	0.67	0.80	0.67	1.00	0.50	0.67	0.88
icu_inter_dominant_uniq	0.75	1.00	1.00	0.67	0.80	0.67	1.00	0.50	0.67	0.88
icu_intra_lhs_high_edge	0.75	1.00	1.00	0.67	0.80	0.67	1.00	0.50	0.67	0.88
icu_intra_lhs_high_uniq	0.75	1.00	1.00	0.67	0.80	0.67	1.00	0.50	0.67	0.88

Table A.2: IND Method on MIMIC: Full Decomposition Evaluation

Case	CC	CF	PP	PR	PF	PCC	EP	ER	EF	ECC
adm_exact_first	0.88	0.94	1.00	1.00	1.00	0.88	1.00	1.00	1.00	0.83
adm_exact_uniqueness	0.88	0.94	1.00	1.00	1.00	0.88	1.00	1.00	1.00	0.83
adm_ratio_first	0.88	0.94	1.00	1.00	1.00	0.88	1.00	1.00	1.00	0.83
adm_ratio_uniqueness	0.88	0.94	1.00	1.00	1.00	0.88	1.00	1.00	1.00	0.83
adm_wentropy_first	0.78	0.80	1.00	1.00	1.00	0.59	1.00	1.00	1.00	0.62
adm_wentropy_uniqueness	0.78	0.80	1.00	1.00	1.00	0.59	1.00	1.00	1.00	0.62
icu_exact_first	0.75	1.00	1.00	0.33	0.50	0.33	0.00	0.00	0.00	0.88
icu_exact_uniqueness	0.75	1.00	1.00	0.33	0.50	0.33	0.00	0.00	0.00	0.88
icu_ratio_first	0.75	1.00	1.00	0.33	0.50	0.33	0.00	0.00	0.00	0.88
icu_ratio_uniqueness	0.75	1.00	1.00	0.33	0.50	0.33	0.00	0.00	0.00	0.88
icu_wentropy_first	0.75	1.00	1.00	0.33	0.50	0.33	0.00	0.00	0.00	0.88
icu_wentropy_uniqueness	0.75	1.00	1.00	0.33	0.50	0.33	0.00	0.00	0.00	0.88

Table A.3: LLM Method on MIMIC: Full Decomposition Evaluation

Case	CC	CF	PP	PR	PF	PCC	EP	ER	EF	ECC
admissions_tbnm	0.33	0.04	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
admissions_clnm	0.97	0.56	1.00	1.00	1.00	0.63	1.00	1.00	1.00	0.50
admissions_cl_samp	0.90	0.89	1.00	1.00	1.00	0.70	0.00	0.00	0.00	0.70
admissions_sch	0.80	0.94	1.00	0.67	0.80	0.56	1.00	0.50	0.67	0.78
admissions_sch_samp	0.80	0.94	1.00	0.67	0.80	0.56	1.00	0.50	0.67	0.78
admissions_prof	0.75	0.87	1.00	0.67	0.80	0.53	1.00	0.50	0.67	0.78
admissions_prof_samp	0.91	0.94	1.00	0.67	0.80	0.64	1.00	0.50	0.67	0.78
icustays_tbnm	0.80	0.83	0.00	0.00	0.00	0.33	0.00	0.00	0.00	0.78
icustays_clnm	0.87	0.83	0.33	0.33	0.33	0.13	0.00	0.00	0.00	0.62
icustays_cl_samp	0.89	0.89	0.33	0.33	0.33	0.33	0.00	0.00	0.00	0.88
icustays_sch	0.89	0.89	0.33	0.33	0.33	0.33	0.00	0.00	0.00	0.88
icustays_sch_samp	0.89	0.89	0.33	0.33	0.33	0.33	0.00	0.00	0.00	0.88
icustays_prof	0.78	0.83	0.33	0.33	0.33	0.33	0.00	0.00	0.00	0.88
icustays_prof_samp	0.78	0.89	0.33	0.33	0.33	0.56	0.00	0.00	0.00	0.88

Table A.4: LLM Method on e-ICU: Full Decomposition Evaluation

Case	CC	CF	PP	PR	PF	PCC	EP	ER	EF	ECC
patient_tbnm	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
patient_clnm	0.83	0.90	1.00	0.75	0.86	0.65	0.50	0.33	0.40	0.97
patient_cl_samp	0.83	0.90	1.00	0.75	0.86	0.65	1.00	0.67	0.80	0.97
patient_sch	0.83	0.90	1.00	0.75	0.86	0.65	1.00	0.67	0.80	0.97
patient_sch_samp	0.81	0.88	1.00	0.75	0.86	0.62	1.00	0.67	0.80	0.97
patient_prof	0.71	0.64	0.40	0.50	0.44	0.62	0.33	0.33	0.33	0.87
patient_prof_samp	0.80	0.88	1.00	0.75	0.86	0.63	1.00	0.67	0.80	0.97
apachePredVar_tbnm	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
apachePredVar_clnm	0.58	0.76	1.00	1.00	1.00	0.06	1.00	0.50	0.67	0.03
apachePredVar_cl_samp	0.80	0.53	0.29	1.00	0.44	0.85	0.17	0.50	0.25	0.13
apachePredVar_sch	0.69	0.70	1.00	1.00	1.00	0.07	1.00	0.50	0.67	0.03
apachePredVar_sch_samp	0.62	0.78	1.00	1.00	1.00	0.36	1.00	0.50	0.67	0.19
apachePredVar_prof	0.83	0.80	0.67	1.00	0.80	0.26	1.00	0.50	0.67	0.27
apachePredVar_prof_samp	0.73	0.81	1.00	1.00	1.00	0.27	1.00	0.50	0.67	0.26

Appendix B

LLM Prompts

B.1 LLM-Based classification

B.1.1 Zero-shot

ROLE: You are a Professional Classifier of Relational Data Tables.

INPUT: A table's input representation

TASK: To analyze the input representation and classify into one of 3 categories: ENTITY, EVENT, or MIXED.

DEFINITIONS:

ENTITY: Entity table is a table that represents things that exist as objects in the real-world. They are not events that happen, but things that exist in tangible or intangible forms.

EVENT: Event table is a table that represents things that happen, typically at a specific time.

They can have a start time and/or an end time. They can have one or many reference columns to an entity.

MIXED: Mixed table is a table that contains BOTH entity-level attributes and event-level information in the same table. They can contain an entity with timestamps and other attributes from different entities. It is a hybrid table that is not a pure entity or a pure event but contains a MIX of attributes which can come from both entity and event tables.

OUTPUT:

Your OUTPUT must be strictly in JSON format with these fields {"reasoning": "...", label: ""}

Where:

"label": Fill this with ENTITY/EVENT/MIXED, after categorizing the table based on the above definitions

"reasoning": Fill this with a text justifying WHY you categorized the table with the specific label.

B.1.2 Few-shot with restriction

ROLE: You are a Professional Classifier of Relational Data Tables.

INPUT: A table's input representation

TASK: To analyze the input representation and classify into one of 3 categories: ENTITY, EVENT, or MIXED.

DEFINITIONS:

ENTITY: Entity table is a table that represents things that exist as objects in the real-world. They are not events that happen, but things that exist in tangible or intangible forms.

EVENT: Event table is a table that represents things that happen, typically at a specific time. They can have a start time and/or an end time. They can have one or many reference columns to an entity.

MIXED: Mixed table is a table that contains BOTH entity-level attributes and event-level information in the same table. They can contain an entity with timestamps and other attributes from different entities. It is a hybrid table that is not a pure entity or a pure event but contains a MIX of attributes which can come from both entity and event tables.

EXAMPLES:

ENTITY: Tables representing Patients, Providers, Locations are all usually

ENTITY tables - BECAUSE they contain entity-level columns representing the object.

EVENT: Tables like labevents, procedures, services, and diagnoses are all usually

EVENT tables BECAUSE they contain event-level columns (e.g., timestamps) representing the event.

MIXED: Tables like admissions are usually MIXED because they contain information about the Event (admission here - has start and end time), but also attributes from the Entity (patient here).

NOTE: Do not rely solely on literal column names or input patterns. Interpret what the input information represents semantically.

OUTPUT:

Your OUTPUT must be strictly in JSON format with these fields {"reasoning": "...", label: ""}

Where:

"label": Fill this with ENTITY/EVENT/MIXED, after categorizing the table based on the above definitions

"reasoning": Fill this with a text justifying WHY you categorized the table with the specific label.

B.2 LLM-Based decomposition

B.2.1 Initial prompt

ROLE: You are a Professional Decomposer of Relational Data Tables.

TASK: Decompose this relational database table profile into semantic clusters (entities and events).

INSTRUCTIONS:

1. Pick the IDENTIFIER columns for the clusters: IDENTIFIER columns are columns whose values uniquely identify a real-world entity or event. Each IDENTIFIER goes into their own clusters.

JSON: Set the "candidate_primary_key" of each cluster with that IDENTIFIER. It should not be null.

2. Consider all the columns which are the NON-IDENTIFIER: You defined N IDENTIFIERS above - so you must have N clusters. For each non-anchor column, assign it to the most semantically similar cluster (represented by the cluster IDENTIFIER). A column can belong to only one cluster.

JSON: Include each non-IDENTIFIER in the "columns" field of that most semantically similar cluster.

3. Create EDGES. An EDGE is a semantic relationship between two groups of columns. A column group is a cluster. Every edge must connect any two previously defined clusters if a relationship exists. Edges can be from an event cluster to an entity cluster. JSON: Define an edge between a from_cluster and a to_cluster

OUTPUT:

Your OUTPUT must be strictly in JSON format with these fields -

Statement within < and > denote Instructions:

```
{{
  "clusters": <list of clusters, each with...> [
    {{
      "cluster_id": <Integer assigned to the cluster, 0-indexed>,
      "columns": <A list of ALL columns belonging to this cluster including the IDENTIFIER>,
      "candidate_primary_key": <this is the IDENTIFIER of this cluster>
    }}
  ],
  "edges": <semantic relationships between clusters, each with...> [
    {{
      "from_cluster": <A cluster ID>,
      "to_cluster": <A cluster ID>
    }}
  ],
  "reasoning": <Fill this with a text justifying WHY you formed these: specific CLUSTERS,
specific IDENTIFIERS, and EDGES>
}}
```

UNIVERSAL RULES:

- Completeness: EVERY column must be in exactly one cluster.
- Only one edge between any two clusters (direction-irrespective).
- No columns can be in more than one cluster.

B.2.2 Validation prompt

ITERATION CONTROL:

You are currently in iteration {iteration_idx + 1} out of {MAX_ITERATIONS}.

This is a refinement step. You may correct structural or semantic mistakes if needed.